



University
of Colorado
Boulder

UNIVERSITY OF COLORADO BOULDER

SVD and NMF comparision on different datasets

Student :

Divya Nallawar
Ainsley Braunscheidel
Reza Naimen
Luke Petet

Teacher :

Prof. James H. Curry

November 20, 2024

Contents

1	Overview of SVD and NMF	2
2	What is Logistic Map?	2
3	Logistic Map Time Series generation	3
4	Time series analysis	6
4.1	Process to apply SSA	6
5	Differences in R and Python	8
6	Wifi Data	8
6.1	In R	9
6.2	In Python	11
7	Pumpkin Data	14
8	Music Data	17
8.1	In Python	18
8.2	In R	18
9	Conclusion	21
10	Biography	22

1 Overview of SVD and NMF

The following paper will make extensive use of both the Singular Value Decomposition (SVD) and Non-Negative Matrix Factorization (NMF). Both of these techniques are primarily used to take a high-dimensional matrix, and reduce the number of dimensions within that matrix. These techniques on matrices are useful in topics such as image processing or natural language processing, with applications on the data such as noise reduction, approximation of areas within an image, feature selection, or compression.

SVD decomposes any matrix $A \in \mathbb{R}^{m \times n}$ into three components as the following:

$$A = U \Sigma V^T,$$

where $U \in \mathbb{R}^{m \times m}$ is a left singular matrix, $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix containing the singular values in decreasing order, and $V^T \in \mathbb{R}^{n \times n}$ is a right singular matrix. Through this decomposition, A can be approximated as:

$$A_{m \times n} \approx U_{m \times r} \Sigma_{r \times r} V_{r \times n}^T,$$

this decomposition is able to reduce the dimensionality of the data and preserve significant features within the data at the same time.

NMF is used when A is a matrix that is composed of only non-negative entries ($A_{ij} \geq 0$). It factorizes A into two non-negative matrices $W \in \mathbb{R}^{m \times k}$ and $H \in \mathbb{R}^{k \times n}$ as the following:

$$A \approx WH.$$

Here, W and H are chosen so that their product is the approximation of A , where the size, k , is smaller than both m and n . From this factorization, every element in both matrices is non-negative, or greater than 0.

The goal of NMF is to make the factorization, WH , as close as possible to an approximation of A . This is achieved through minimizing the error between A and WH , which would be the difference between the matrices. This is expressed as:

$$\min_{W, H} \|A - WH\|_F^2,$$

where $\|\cdot\|_F$ represents the magnitude of error between the original and the approximation given by NMF through using the Frobenius norm. Through this error check, we see that all entries in W and H must be non-negative.

2 What is Logistic Map?

The logistic map[5] is an example of how simple nonlinear dynamical systems can exhibit chaotic behavior. This model shows how the output changes based on initial conditions.

The equation for the logistic map is given as:

$$x_{n+1} = r \cdot x_n \cdot (1 - x_n)$$

Where:

- x_n : The value at iteration n (between 0 and 1), often representing the ratio of the current population to the maximum possible population.

- r : The control parameter (positive real number) that determines the growth rate.
- x_{n+1} : The value at the next iteration.

The logistic map exhibits different behaviors depending on the value of r :

- For $0 < r < 1$: The population dies off and goes to zero.
- For $1 < r < 3$: The population stabilizes at a fixed point.
- For $3 < r < 3.57$: The population oscillates between two or more values (periodic behavior).
- For $r > 3.57$: The system becomes chaotic, with seemingly random and unpredictable behavior.
- For specific values of r (like around $r = 3.83$), the system exhibits periodic cycles again (windows of stability) amidst chaos.

3 Logistic Map Time Series generation

For this report, some of the initial conditions used to generate the logistic map time series are:

- $r = 4$
- initial values = 0.4
- Number of iterations = 10000

Logistic map equation for given $r=4$:

$$x_{n+1} = 4 \cdot x_n \cdot (1 - x_n)$$

Logistic map generated for the above conditions is shown by the figure 1.

Observation: It can be seen in the figure 1, for a given time between 70 to 90, it is observed that the population is almost zero, but not zero as shown in figure 2.

Hypothesis: For a given logistic map equation $r \cdot x_n \cdot (1 - x_n)$, when subsequent values of x_n are equal to $(1 - x_{n-1})$ then logistic map follows a pattern involving this value, it indicates that the logistic map has entered a periodic loop

$$x_n = (1 - x_{n-1})$$

from the logistic map equation:

$$x_n = r \cdot x_{n-1} \cdot (1 - x_{n-1})$$

Substituting x_n value in the above equation:

$$(1 - x_{n-1}) = r \cdot x_{n-1} \cdot (1 - x_{n-1})$$

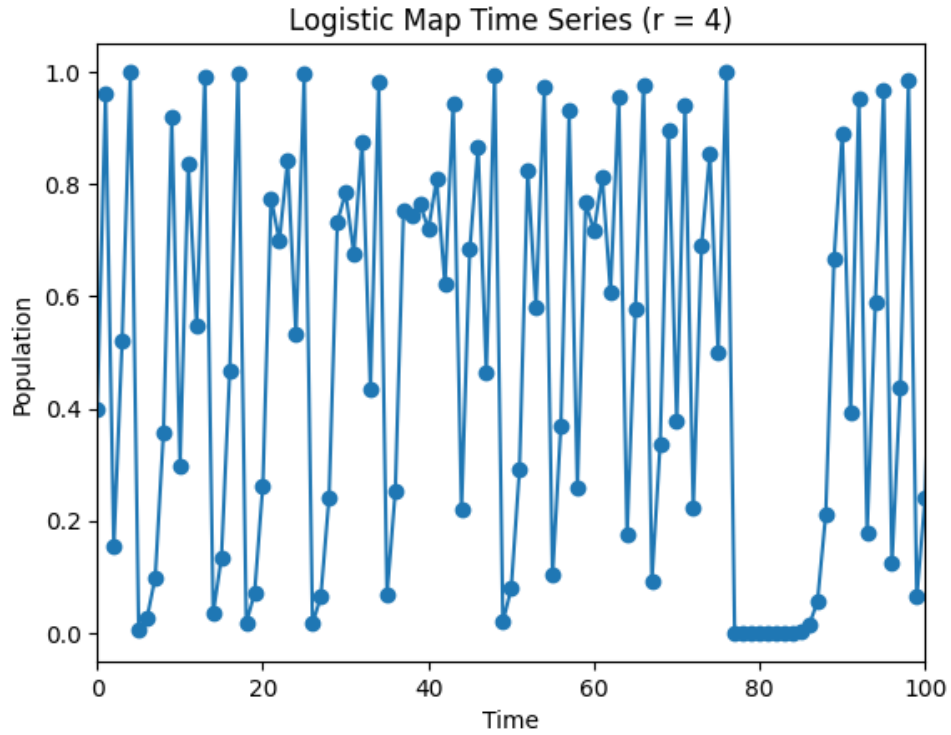


Figure 1: Logistic map for given initial condition as 0.4 and r as 4

$$x_{n-1} = 1/r \quad \text{or} \quad x_{n-1} = 1$$

Thus, in logistic map, the values of the state becomes $1/r$ then the logistic map enters a periodic loop. This has been observed for $x_0 = 1/4$ or $x_0 = 1$ in our case, as r value is equal to 4. The results for this has been visualized in figure 3. Using the initial condition $x_0 = 0.5$ and $r = 4$, the logistic map was generated for multiple iterations. The first value is 0.5, followed by 1 for the second value, and then all subsequent values settle to 0 for the rest of the iterations. This behavior continues regardless of the number of iterations, illustrating a rapid decay to zero after the initial fluctuations.

```
time_series[70:100]
✓ 0.0s

array([3.78511676e-01, 9.40962348e-01, 2.22208829e-01, 6.91328262e-01,
       8.53573985e-01, 4.99941749e-01, 9.99999986e-01, 5.42910909e-08,
       2.17164352e-07, 8.68657218e-07, 3.47462586e-06, 1.38984551e-05,
       5.55930478e-05, 2.22359829e-04, 8.89241541e-04, 3.55380316e-03,
       1.41646946e-02, 5.58562240e-02, 2.10945225e-01, 6.65789348e-01,
       8.90055568e-01, 3.91426615e-01, 9.52847280e-01, 1.79717363e-01,
       5.89676130e-01, 9.67832767e-01, 1.24530009e-01, 4.36089143e-01,
       9.83661610e-01, 6.42857895e-02])
```

Figure 2: Data point values for the flat surface in plot 1

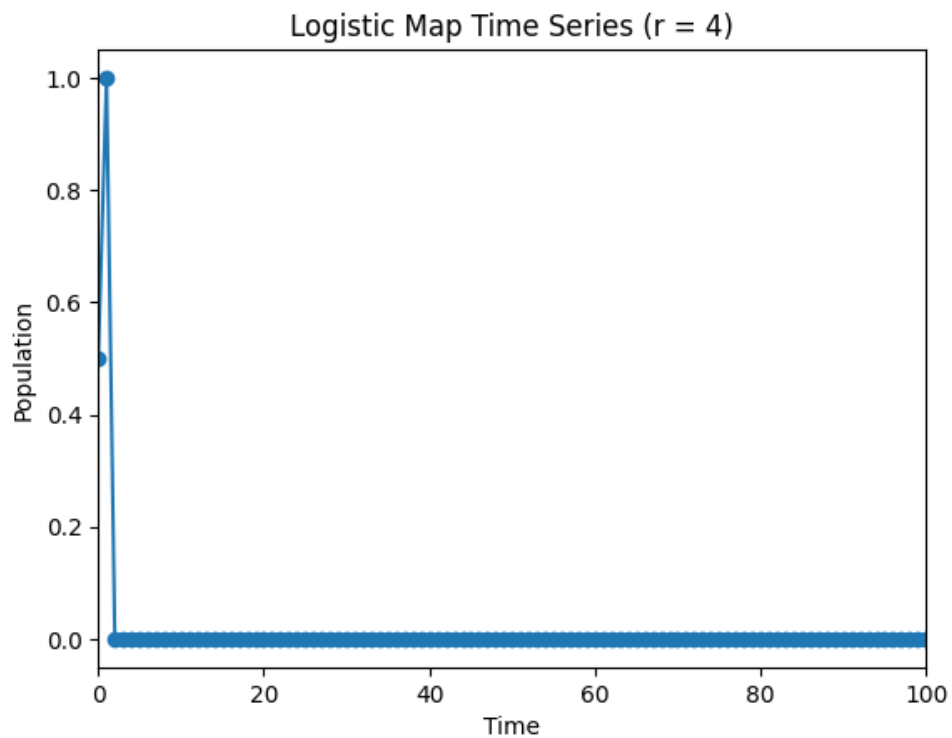


Figure 3: Logistic map for given initial condition as 0.5 and r as 4

4 Time series analysis

The meaningful information from a time series is very difficult to extract, since the time series data often a combination of complex patterns. One of the effect ways of analyzing and decomposing such data is Singular Spectrum Analysis(SSA)[8, 6].

SSA is singular value decomposition (SVD) based method used for analysis of time series data. SSA is a technique used to decompose a time series into interpretable components such as trends, oscillatory patterns, and noise.

SSA applied to a logistic map can enable the identification and subsequent separation of the different dynamics present within time series data. In spite of the simplicity of this logistic map, it is capable of yielding chaotic behavior depending on parameters, an illustration can be seen above in figure 1. Such dynamics are systematically showed using SSA, which extracts the pattern from the time series.

4.1 Process to apply SSA

Time series data is typically one-dimensional, with values depending on a single variable (such as time or another sequential variable). In the case of logistic time series data, the data is generated as a sequence based on one variable, making it inherently one-dimensional.

To analyze and extract meaningful patterns from this one-dimensional time series data, Singular Value Decomposition (SVD)[11, 4] cannot be applied as it takes a 2-dimensional data as an input. However, to perform SVD, the data must first be transformed into a matrix format. This is acheived by embeddings step in SSA .

Embedding is the procedure of transforming the vector Y into a trajectory matrix X which is also known as **Hankel Matrix**. This process arranges the data points into a structured format that allows SSA to perform effectively. It involves creating overlapping segments of the original data, which then become rows or columns of the matrix, depending on the chosen window size. This process enables SSA to identify and extract the underlying patterns in the data.

The trajectory matrix is given as:

$$\mathbf{X} = [\mathbf{X}_1 \cdots \mathbf{X}_K] = (x_{ij})_{i=1,j=1}^{L,K} = \begin{bmatrix} x_1 & x_2 & x_3 & \cdots & x_K \\ x_2 & x_3 & x_4 & \cdots & x_{K+1} \\ x_3 & x_4 & x_5 & \cdots & x_{K+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_L & x_{L+1} & x_{L+2} & \cdots & x_N \end{bmatrix}$$

where K is the length of the sliding window, and L represents the number of windows generated from the original time series.

This trajectory matrix X is then decomposed using the SVD into three matrices:

$$X = U\Sigma V^T$$

The singular values and corresponding vectors are then grouped to separate different components of the time series, such as trends or periodic signals. In this step decisions

are made to understand which components to retain based on their significance, often visualized using a scree plot [7]

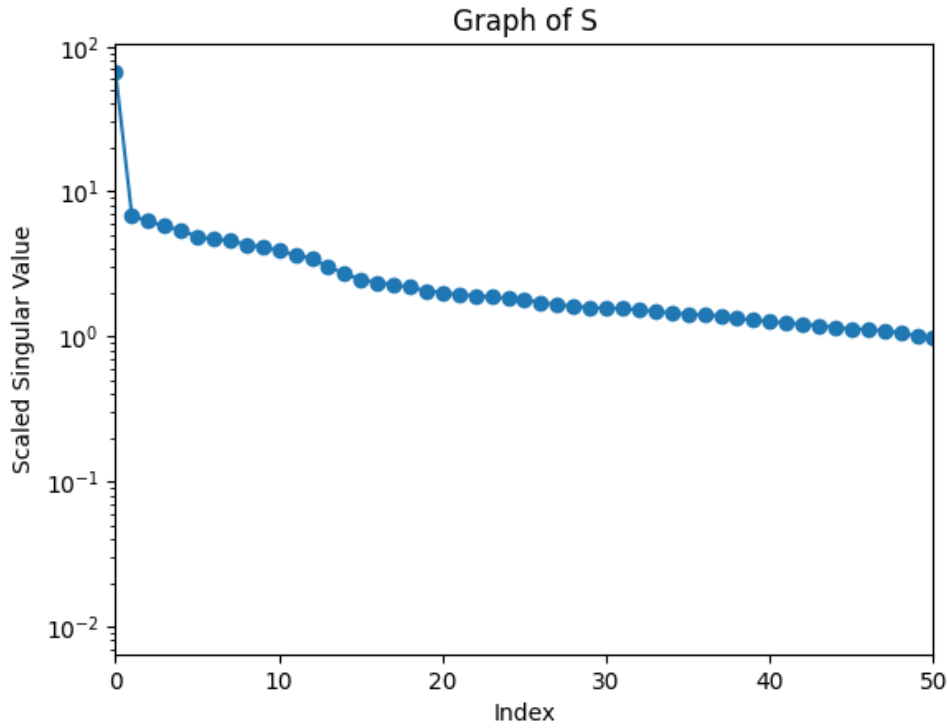


Figure 4: The figure shows scaled singular values for the logistic map data in python

The grouped components are combined to reconstruct the time series, resulting in isolating trends, seasonal patterns, or noise as needed.

The figure 7 illustrates the singular values obtained from applying Singular Spectrum Analysis (SSA) to the Wi-Fi data collected from the Williams Village East building at the University of Colorado Boulder. For this analysis, a window length of 344 was chosen, which corresponds to half of the data series length. The plot displays the first 100 eigenvalues, showing the significance of different components in the decomposition of the original time series.

Given a time series of length N and a chosen `window_length`, the number of columns K in the trajectory matrix is calculated as

$$K = N - \text{window_length} + 1$$

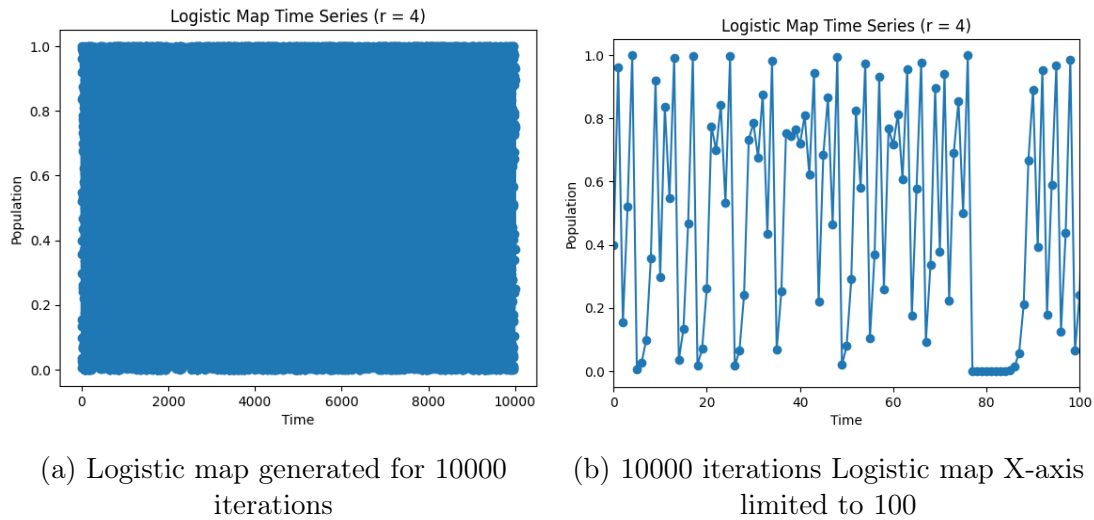


Figure 5: Logistic map time series generated at $r = 4$ and initial value as 0.4 for 10,000 iterations. Part (a) shows the full 10,000 iterations, while part (b) shows a limited version of the x-axis for the same 10,000 iterations for better visibility.

5 Differences in R and Python

While Python is a language designed for general-purpose programming, R is explicitly designed for statistical and data analysis. Due to this R, has a robust set of internal packages that relate to data visualization and modeling, while Python often requires importing additional packages for the same set of features. For instance, base R includes the function `svd()` which is capable of performing Singular Value Decomposition of a matrix. In Python, there is no base feature to perform SVD, and the numpy package must be utilized for the `numpy.linalg.svd()` function. For analysis purposes, especially relating to images or music, R is under equipped to deal with these forms of data streams. Python, however, has many packages that make it easy to import images or music and perform manipulations on them with only a few lines of code. Therefore, the differences between R and Python are that for traditional statistical analysis or data science, and data visualization, R is often the best choice. For analysis of non-numeric forms of data and additional tasks beyond statistical analysis, Python is the better choice.

6 Wifi Data

The WiFi data accessed from various buildings at the University of Colorado Boulder resembles the time series generated by the logistic map. To analyze the decomposition of this time series, examined data from the William Village East (WVE) building for October 2023. In the figure 6, the x-axis represents the 31 days of October, while the y-axis shows the count of devices connected to a specific WiFi access point in the WVE building.

6.1 In R

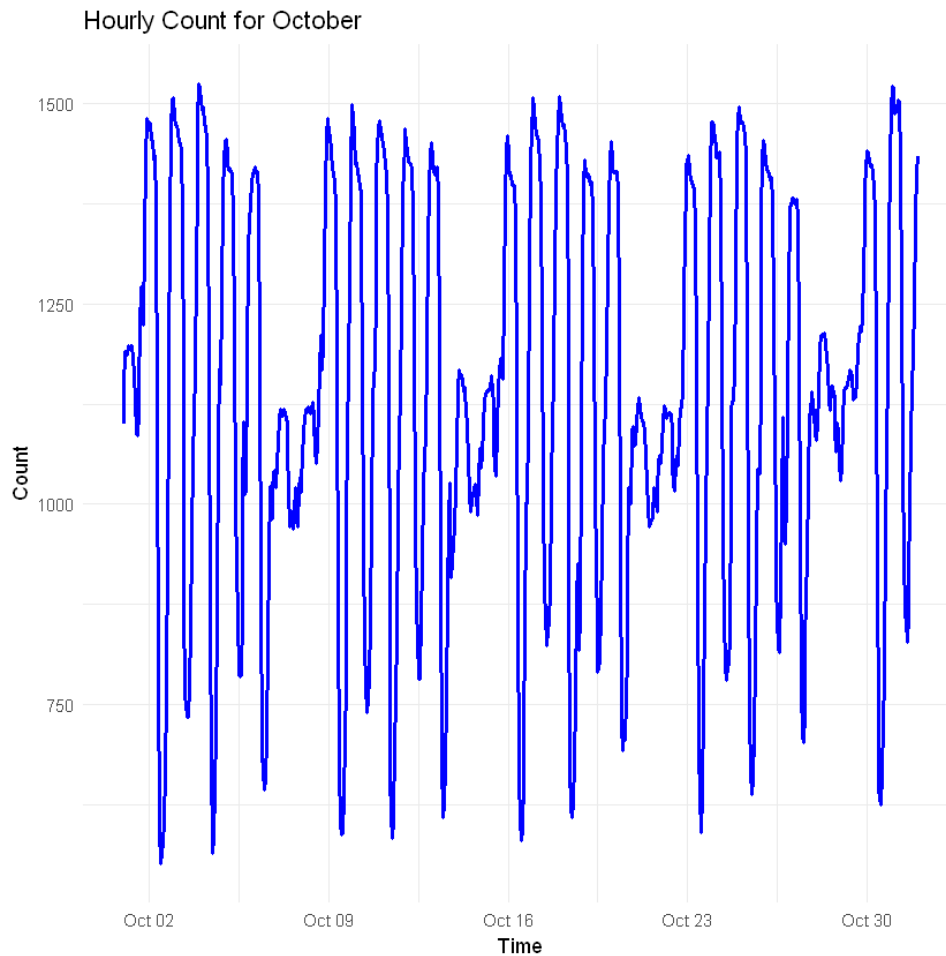


Figure 6: The figure is the time-series representation of Wi-fi data for Williams Village East(WVE) building for October month.

Observation: In the figure 7, it is observed that the singular values for the WiFi data from the WVE building exhibit small steps, indicating that the singular values are closely spaced.

In SSA, pairs of nearly equal eigenvalues are characteristic of oscillatory patterns. However, due to numerical and sampling errors, the presence of paired eigenvalues alone does not conclusively confirm the detection of an oscillation.[1]

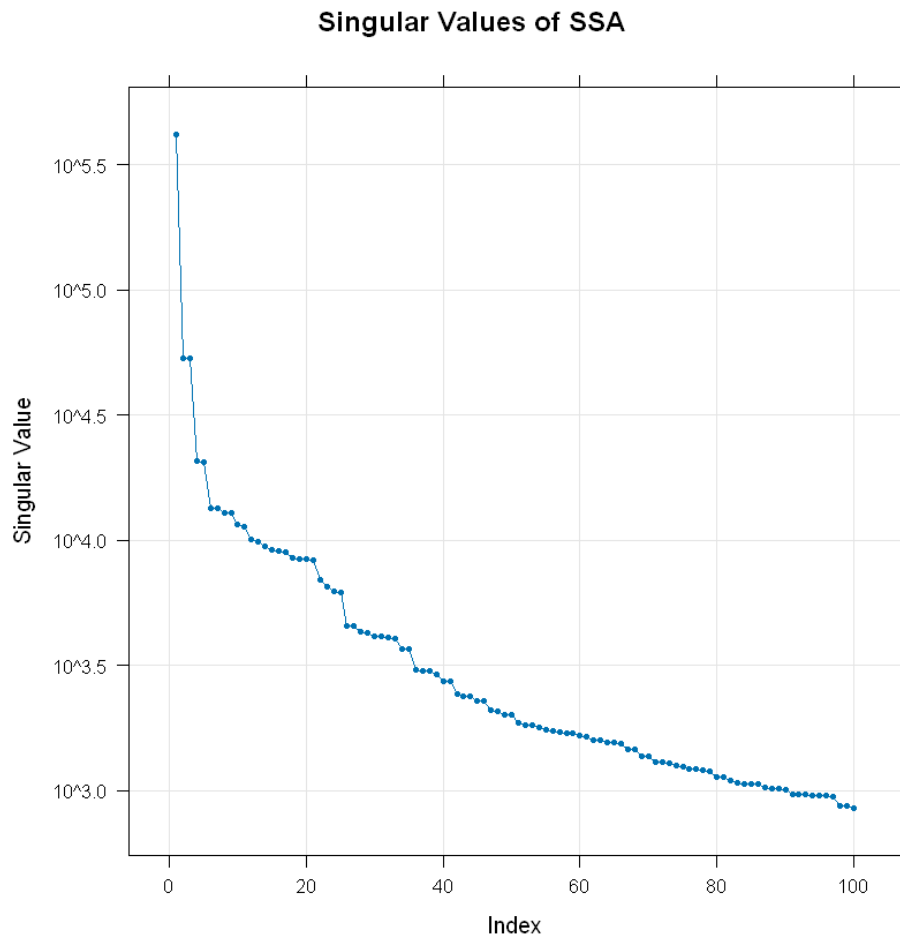


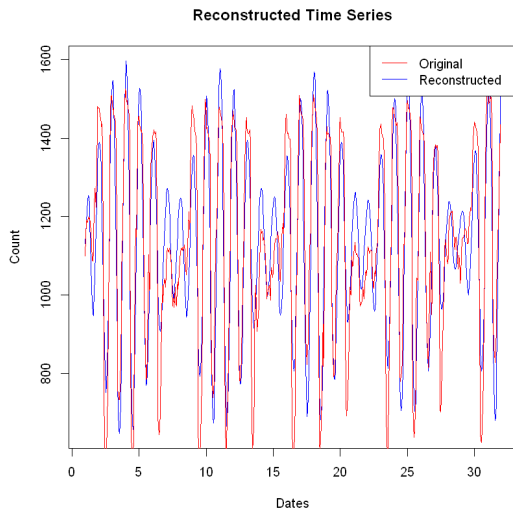
Figure 7: The figure shows singular values for the wifi data (williams village building)

The figures 8 display the **reconstructed time series** of WiFi data collected from the WVE building on campus. SSA was applied to the data using a **window size of 372** (half the length of the dataset) and **100 eigenvalues**

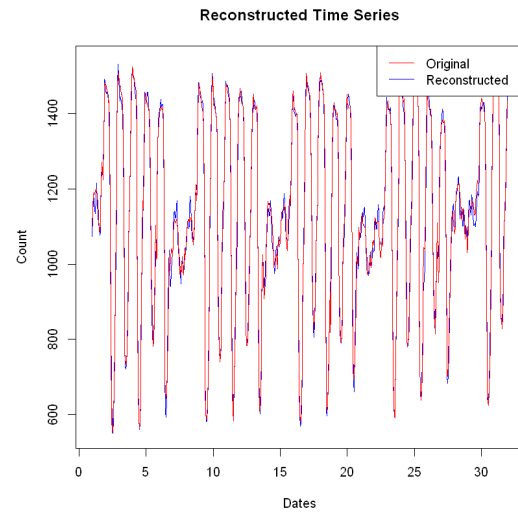
In the figure 10a, the reconstruction used the **first 7 eigenvalues**. Although the general shape and pattern of the original series were retained, there are noticeable discrepancies at certain points where the reconstructed series fails to fully align with the original data points. Due to the singular value pairs shown in figure 7, the reconstruction captures the oscillations presented by the original time series.

In the figure10b, the reconstruction was performed using **50 eigenvalues**. This version shows a significant improvement, capturing most of the original data's behavior with higher precision. The reconstructed series aligns more closely with the original, indicating that including more eigenvalues helps to better model the variations in the time series.

From the figure 9 and figure 10, it is observed that the trend of the series is fully captured but the magnitude is low, it suggests that the Non-Negative Matrix Factorization (NMF) process is successful in identifying the underlying structure or patterns of the data (the "shape" or "trend"), but the scaling or normalization of the factors is affecting the absolute values.



(a) Reconstructed time series Wifi data(WVE) for first **7** values of wifi data in blue + original WVE wifi time series in red



(b) Reconstructed time series Wifi data(WVE) for first **50** values of wifi data in blue + original WVE wifi time series in red

Figure 8: SVD Reconstruction of WiFi Data

NMF algorithms focus on minimizing overall reconstruction error , not on preserving the absolute scale of the original data.[3]

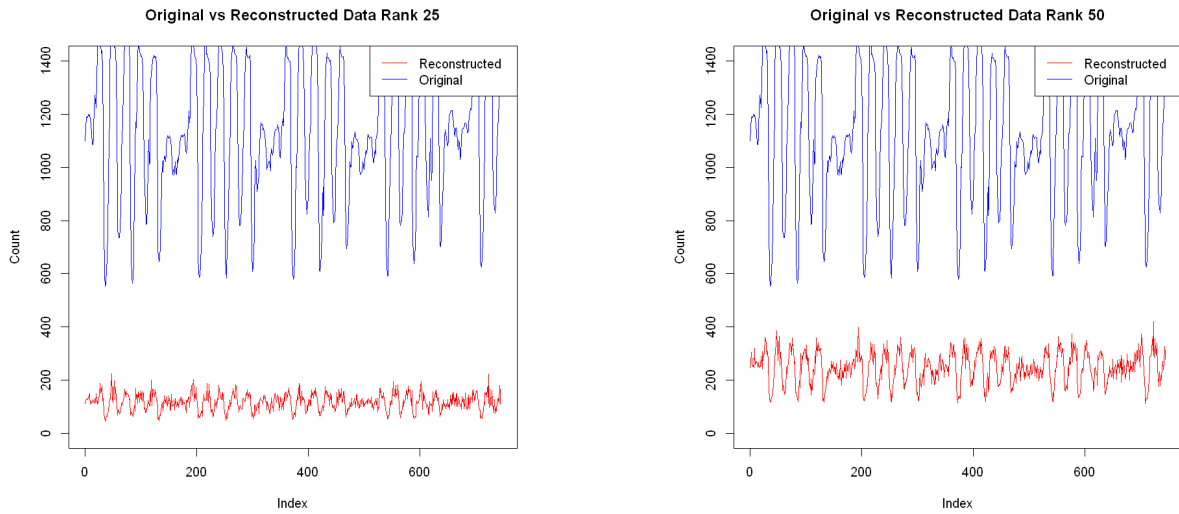
6.2 In Python

This section shows a comparison of the wifi data of WVE building on campus using 2 matrix factorization methods a) Singular value decompsioiton(SVD) and b) Non-negative matrix factorization(NMF)

Using the exisiting packages like ‘SVD’ from ‘numpy.linalg’ and ‘NMF’ from ‘sklearn.decomposition’ in python, performed these methods and reconstructed for differ-ent rank like 25, 50, 75, 150.

The figures 11 display the **SVD reconstruction of time series** of WiFi data col-lected from the WVE building on campus. SSA was applied to the data using a **window size of 372** (half the length of the dataset, which was 744). It can be observed that even though lower rank approximation like rank 25 captures the pattern of the data, a bit higher rank than 25 [11a], like rank 150 [11c] captures most of the data with less mean square error andmean absolute error as shown in 1.

The figures 12 display the **NMF reconstruction of time series** of WiFi data collected from the WVE building on campus. Using the similar data used for SVD re-construction. It can be observed that even though lower rank approximation like rank 25 [12a] captures the pattern of the data, a bit higher rank than 25, like rank 150 [12c] captures most of the data with less mean square error and mean absolute error as shown



(a) Reconstructed time series Wifi data(WVE) for first **25** values

(b) Reconstructed time series Wifi data(WVE) for first **50** values

Figure 9: NMF Reconstruction of WiFi Data (rank 25 and 50)

in 1.

Rank Approximation	MSE		MAE	
	SVD	NMF	SVD	NMF
25	2798.967	2967.330	40.602	42.195
50	865.560	1083.507	22.550	24.177
75	423.904	535.324	15.535	17.480
150	81.835	137.267	6.990	8.999

Table 1: Comparison of MSE and MAE for different rank approximations using SVD and NMF

The table 1 shows the mean square error(MSE) and the mean absolute error(MAE) between the reconstructed and the original time series for different rank i.e., 25,50, 75, 150 using 2 different methods SVD and NMF.

The table clearly shows that the SVD outperforms the NMF. The MSE and MAE for SVD is less than the MSE and MAE for NMF.

SVD provides an optimal low-rank approximation of a matrix by minimizing the Frobenius norm, which is closely related to minimizing the MSE. This means that SVD is mathematically designed to capture the most significant variance in the data with fewer components, leading to lower reconstruction errors like MSE and MAE. This property makes SVD more effective at reducing error compared to NMF, which is constrained by non-negativity. This is discussed in [9], where it is noted that SVD tends to conflate different components but still provides a strong approximation.

NMF imposes a non-negativity constraint on the factorized matrices, which can limit its ability to approximate data as accurately as SVD in some cases. NMF is particularly useful for interpretability but may result in higher errors when approximating matrices

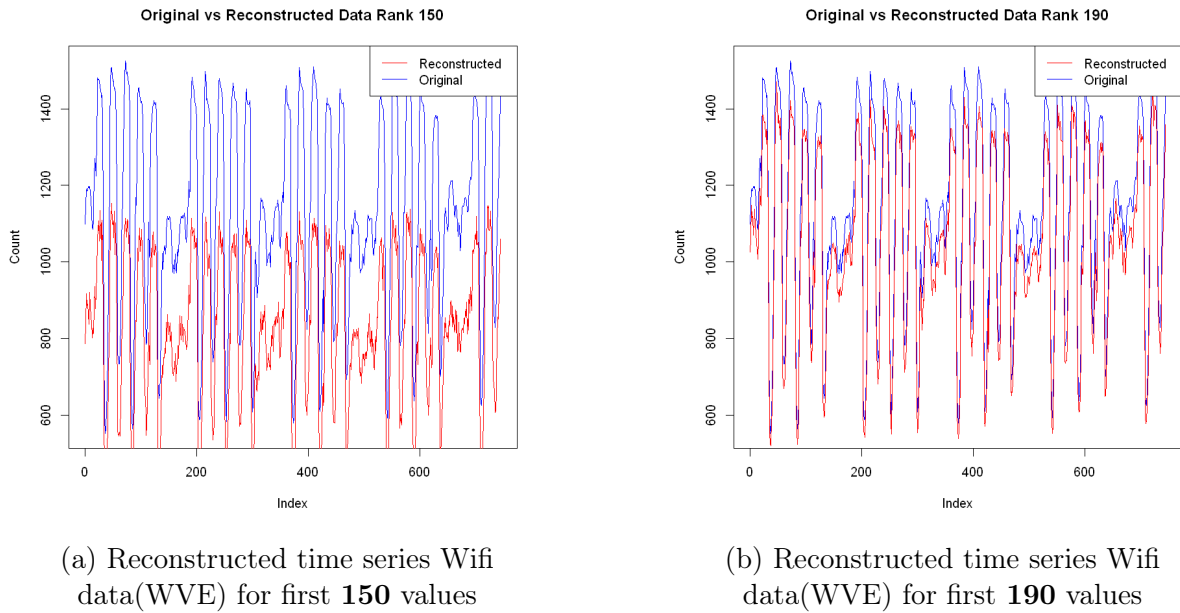


Figure 10: NMF Reconstruction of WiFi Data (rank 150 and 190)

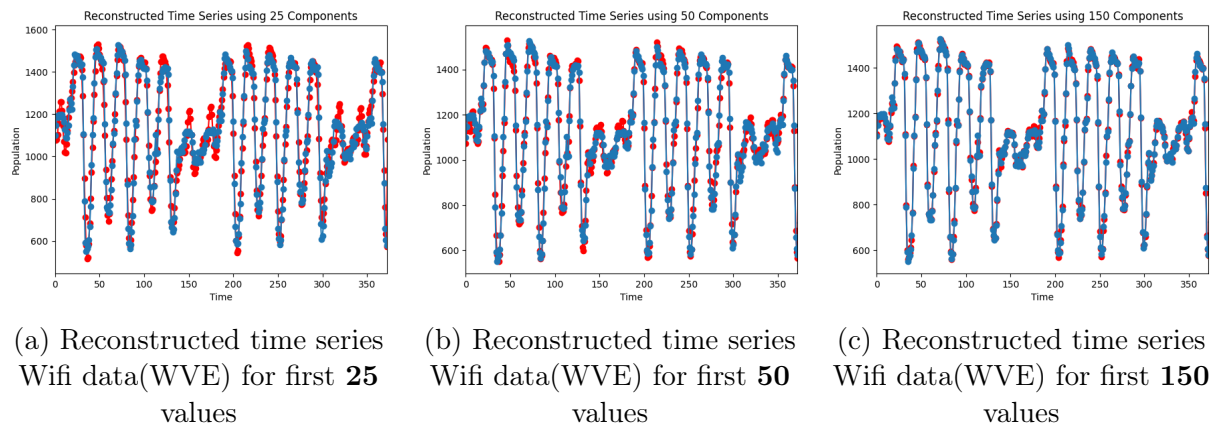


Figure 11: SVD Reconstruction of Wifi time series data

that contain both positive and negative values or when the structure of the data does not align well with non-negative representations. This constraint leads to higher MSE and MAE, as noted in [10], where it highlights that NMF is more interpretable but less flexible compared to SVD.

While both methods are useful for dimensionality reduction, SVD typically outperforms NMF in terms of reconstruction accuracy (i.e., lower MSE and MAE). This is because SVD can handle any matrix type, whereas NMF is restricted to non-negative matrices, limiting its approximation capabilities.[2]

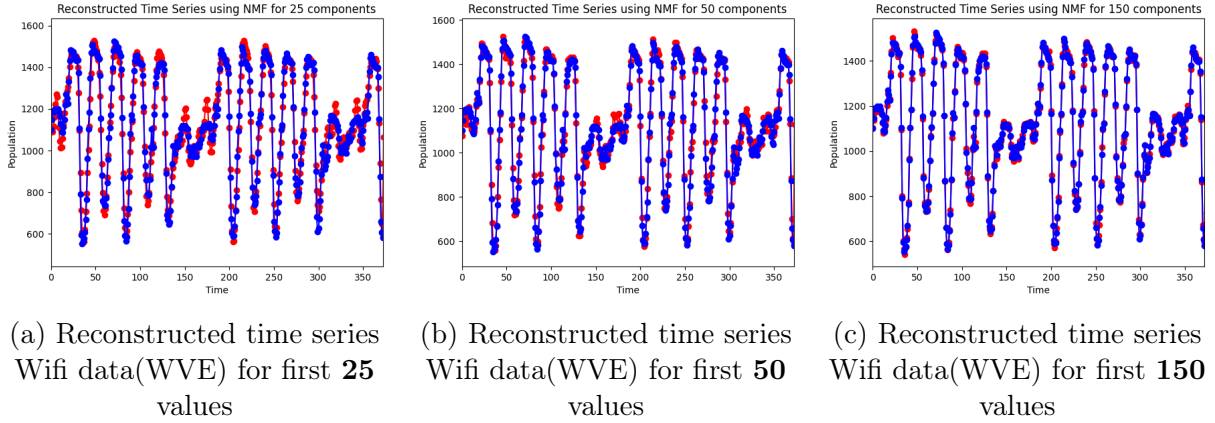


Figure 12: NMF Reconstruction of Wifi time series data

7 Pumpkin Data

Singular Value Decomposition (SVD) and Nonnegative Matrix Factorization (NMF) are powerful mathematical tools that are widely used in the field of data science, especially for dimensionality reduction. One of their most important and practical applications is in image compression, where each decomposition decomposes the image into smaller matrices that can be used to reconstruct the original image while significantly reducing the original image size.

During our analysis, we examined an image of a pumpkin18 in conjunction with the Halloween season. The image consists of 660 x 948 pixels with three color channels: red, green, and blue. We utilized the NumPy library in Python to convert the image into a matrix for the purposes of Singular Value Decomposition (SVD) and Nonnegative Matrix Factorization (NMF).

We then performed SVD on the four (green color channel, red color channel, blue color channel, and gray scale color channel) matrices of the original matrix. Each decomposition resulted into three decomposed matrices:

$$\begin{aligned} A_G &= U_G \Sigma_G V_G^T, \\ A_R &= U_R \Sigma_R V_R^T, \\ A_B &= U_B \Sigma_B V_B^T, \\ A_{GS} &= U_{GS} \Sigma_{GS} V_{GS}^T. \end{aligned}$$

In SVD the singular values are contained in the Sigma matrix. The singular values are non-negative and are usually arranged in descending order along the diagonal. The plot of the singular values of all four decompositions can be seen below.

Table below shows the top 10 singular values for each of the decompositions.



Figure 13: Original pumpkin image of size 660 x 948 pixels

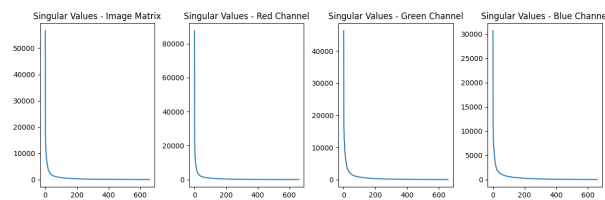


Figure 14: Image caption

Table 2: Singular Values for Each Color Channel

Index	Gray Scale	Red Channel	Green Channel	Blue Channel
0	56619.84	87655.46	46352.44	30691.37
1	17019.04	22916.47	15912.60	11475.34
2	15707.69	19995.05	15471.44	10421.30
3	12488.45	15346.17	12453.83	7857.76
4	11276.42	13015.64	11701.30	7497.88
5	10654.55	11896.69	11490.12	6978.38
6	9616.96	10032.93	10160.88	6368.26
7	8666.36	9836.59	9023.28	5927.07
8	7767.98	8697.47	8277.72	5513.82
9	7591.82	8142.48	8261.74	5273.76

Based on the table of singular values, we can observe that there is a significant difference between the singular values across the different color channels. Notably, the red color channel has the highest singular values. This suggests that the red color channel captures the most information about the structure and the content of the image.

We used the following ranks to reconstruct the pumpkin image.

We used rank 50 to reconstruct the image, the reconstructed image using rank 50 is shown below.

Table 3: Ranks

Rank

10

25

50

100



Figure 15: Reconstructed Image using Rank 50

We then applied NMF to the pumpkin picture, we used 50 components to reconstruct the image. The reconstructed image of the original pumpkin using 50 components is shown below.



Figure 16: Reconstructed Pumpkin Image using NMF ($K = 50$)

We evaluated the performance of both methods using the Frobenius norm. The results are presented in the table below.

Table 4: Comparison of Frobenius Norms

Method	Frobenius Norm
NMF	251013.54
SVD	237095.45

Our results show that SVD achieved better image reconstruction compared to NMF, as indicated by the lower Frobenius norm. However, visually, the reconstructed image quality of NMF appears better than the reconstructed image of SVD. This discrepancy suggests that while SVD captured the most important information more efficiently, NMF may have preserved more nuanced details.

8 Music Data

Our chosen music for this analysis was the piece "Merry-Go-Round of Life" by Joe Hisaishi. We took the audio file that the music was stored on and were able to turn it into a time series data set, and were able to visualize it by plotting the data in both R and Python languages. In the plot we have the x-axis representing the sample index and the y-axis representing the amplitude points the song reaches throughout its first minute of run time.

8.1 In Python

This section demonstrates the process taken to reconstruct an audio file in Python. Using the libraries 'librosa' and 'numpy' to turn the audio file into time series data and then deconstructing that data using SVD and NMF.

Figure 14 displays the full audio file turned into the time series data that we will base the following deconstruction on.

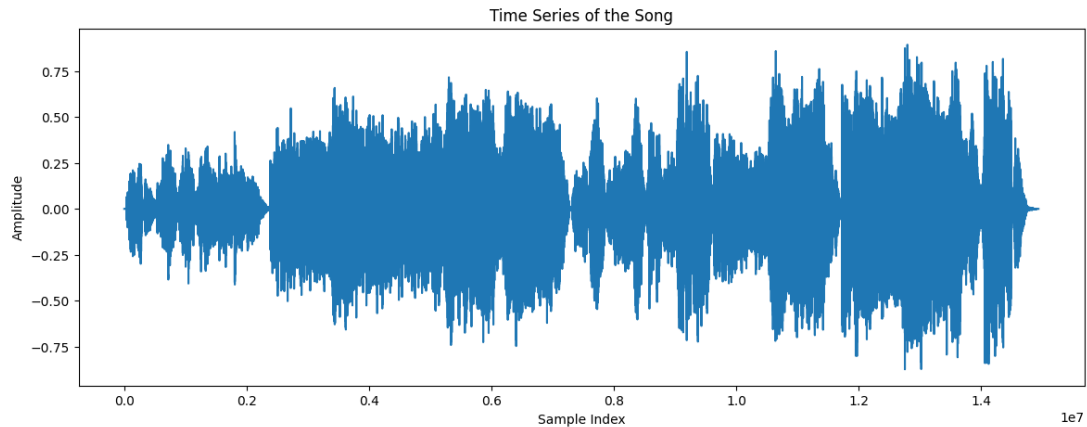


Figure 17: Time Series plot of full music piece

Figure 15 shows the time series adjusted to the first minute of the audio file.

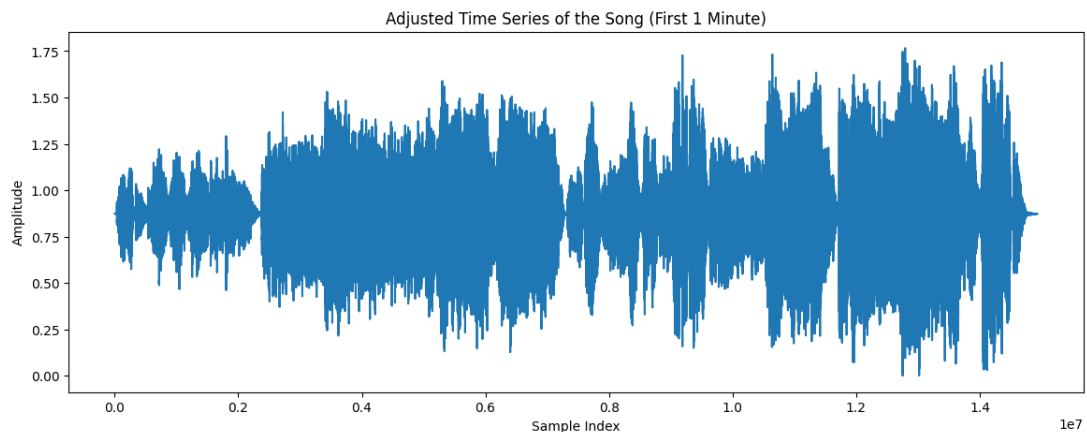


Figure 18: Time series plot adjusted to the first minute

Figure 16 is the plotted time series after using Python to deconstruct the data using Non-Negative Matrix Factorization.

8.2 In R

The process of displaying an audio file as a time series is similar in R - the processes for R importing mp3 files as an internal package make it impossible to perform NMF on this

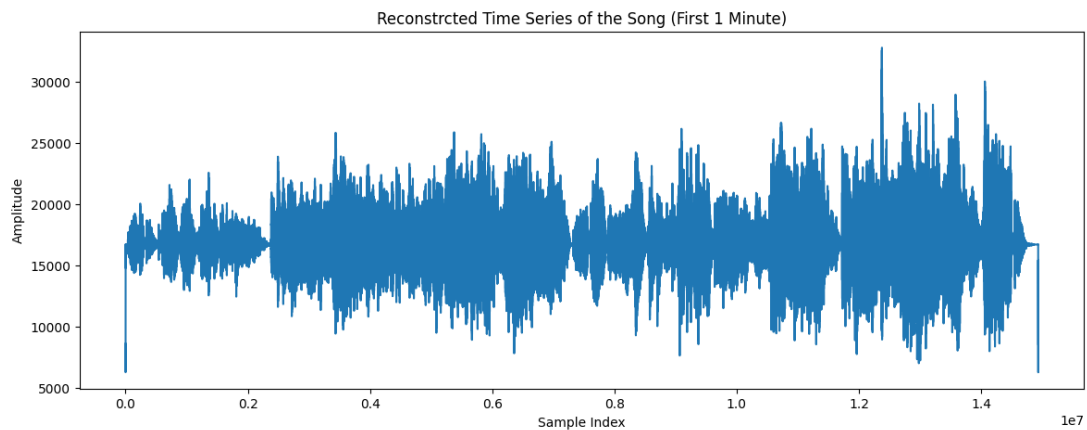


Figure 19: Reconstruction of the time series plot

time series, however. This is because the processes of storing the amplitude within R involves negative elements, which NMF cannot be performed on. However, we can decompose the audio file into components, and using the absolute value of these, we can use NMF and display the result of these modified components over time. This is the only way to display NMF on an audio file with limitations of R.

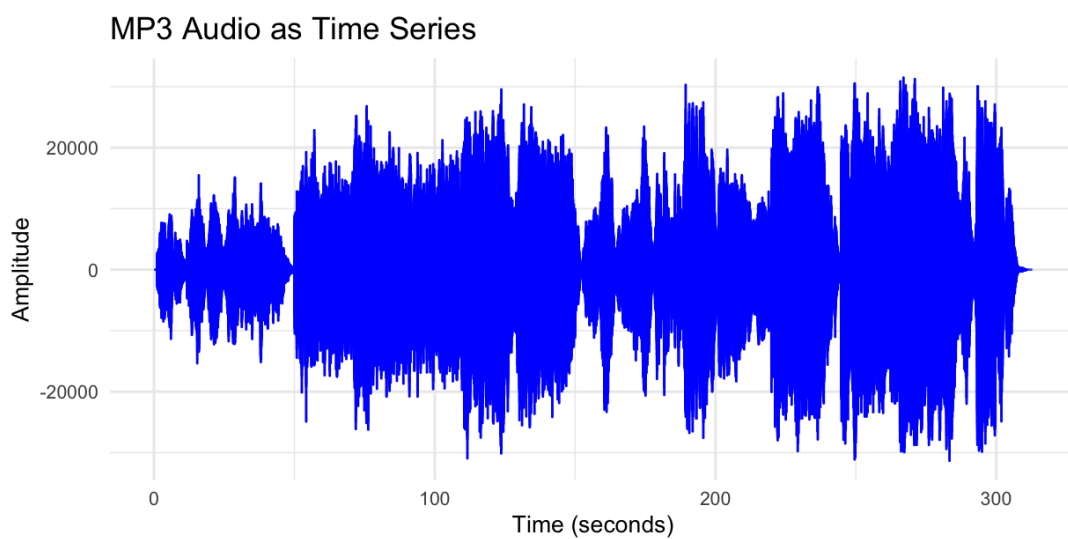


Figure 20: Time Series plot of full music piece

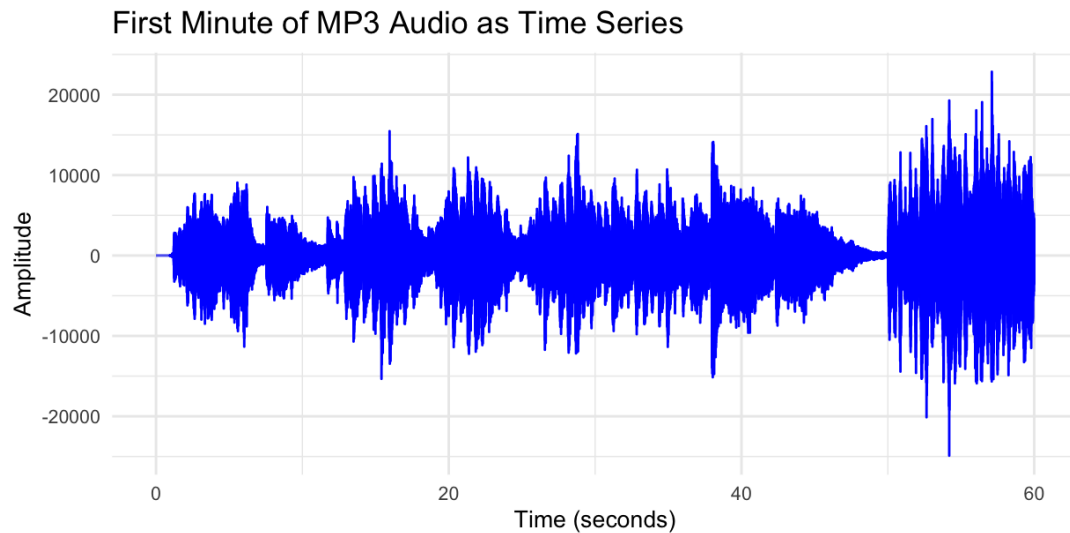


Figure 21: Time series plot adjusted to the first minute

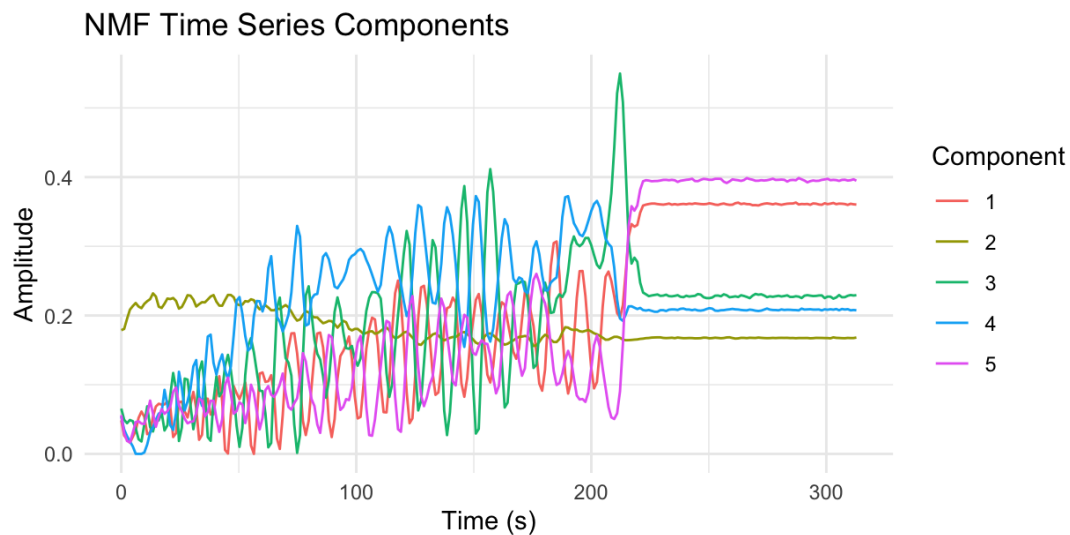


Figure 22: New time series plot on 6 components with NMF performed on them from the full audio file in R. Note that in R, NMF on an audio file does not act as a form of compression, rather, it is used to isolate individual components of the file into respective areas of decomposition. Here, we can see all 6 forms that the NMF decomposed the audio into.

9 Conclusion

The logistic map exhibits periodic behavior when certain conditions are met, specifically when subsequent values of x_n stabilize at $1/r$. This indicates that the system has entered a periodic loop, where the population oscillates between specific values. The report demonstrates that for $r = 4$, when the initial condition $x_0 = 1/4$ or $x_0 = 1$, the system quickly settles into a periodic loop, confirming the hypothesis that when $x_n = 1/r$, the logistic map enters a stable periodic cycle.

In this paper, we conducted a comparative study of SVD and NMF, applying them to various datasets. Both techniques are essential in data analysis and dimensionality reduction. While they are powerful tools in data science, they differ significantly in their approaches. As noted in Lee and Seung's work, NMF is a learning-based decomposition method with a non-negativity constraint, limiting its application to non-negative matrices. In contrast, SVD is a more versatile technique, applicable to a wide range of data without such limitations.

For Wi-Fi data, the SVD reconstruction of the time series performs well with a 50-rank approximation, achieving a decent balance between pattern fidelity and amplitude matching. In contrast, NMF effectively captures the overall pattern of the time series even with a low-rank approximation (e.g., rank 25). However, to match both the pattern and the amplitude accurately, a higher rank approximation is required for NMF (e.g., rank 190 in this case).

To conclude, Although NMF's restriction to non-negativity is perfectly suited for datasets like images, where pixel intensities are inherently non-negative, SVD can introduce negative values in its reconstruction. This can lead to interpretability issues or errors, especially when dealing with data that must strictly remain non-negative. NMF, by preserving non-negativity, ensures that the reconstructed components are both physically meaningful and consistent with the original data's characteristics, making it a more appropriate choice for applications like image processing. However, in scenarios where non-negative values are not a strict requirement, SVD outperforms NMF in accuracy making it the preferred choice for such use cases.

References

- [1] Atmospheric and UCLA Oceanic Sciences Department. *Singular Spectrum Analysis*. n.d. URL: <https://dept.atmos.ucla.edu/tcd/singular-spectrum-analysis>.
- [2] Chelsea Boling and Kumer Das. "A Comparison of SVD and NMF for Unsupervised Dimensionality Reduction". In: Oct. 2014.
- [3] Jean-Philippe Brunet et al. "Metagenes and molecular pattern discovery using matrix factorization". In: *Proceedings of the national academy of sciences* 101.12 (2004), pp. 4164–4169.
- [4] L. Brunton and J. N. Kutz. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge, 2022.

- [5] Wikipedia contributors. *Logistic map*. https://en.wikipedia.org/wiki/Logistic_map. 2024.
- [6] Wikipedia contributors. *Singular spectrum analysis*. https://en.wikipedia.org/wiki/Singular_spectrum_analysis. 2024.
- [7] Cheng Deng. *Time series decomposition using singular spectrum analysis*. 2014.
- [8] James B Elsner and Anastasios A Tsonis. *Singular spectrum analysis: a new tool in time series analysis*. Springer Science & Business Media, 2013.
- [9] Paul Fogel et al. “A tale of two matrix factorizations”. In: *The American Statistician* 67.4 (2013), pp. 207–218.
- [10] Ben Moore. *Singular Value Decomposition (SVD) vs Non-Negative Matrix Factorization (NMF): Difference*. Accessed: 2024-11-11. URL: <https://benmoore.dev/singular-value-decomposition-svd-vs-non-negative-matrix-factorization-nmf-difference/>.
- [11] Wikipedia, ed. *Singular value decomposition*. 2024. URL: https://en.wikipedia.org/wiki/Singular_value_decomposition.

10 Biography

- **Ainsley Braunscheidel** is a Statistics and Data Science undergraduate student at University of Colorado, Boulder. She has experience in coding languages such as R and Python, as well as various techniques for data analysis.
- **Divya Nallawar** was born in India and completed her high school and undergrad in India and now is a Data Science graduate student at University of Colorado, Boulder. She has experience in software development, data analysis, and machine learning through different roles at Amdocs, StateSimplify LLC, and the University of Colorado Boulder.
- **Luke Petet** is a Data Science undergraduate student at the University of Colorado, Boulder. He has experience in data analysis with an emphasis in the application of data analysis on audio engineering.
- **Reza Naiman** was born in Kabul, Afghanistan, and moved to Englewood, Colorado, in 2016. After graduating from Englewood High School, he continued his education at the University of Colorado Boulder. He is a senior in the Statistics and Data Science program with a minor in Computer Science. He has completed two internships with Frontier Technologies Defense as a data analyst and software developer. His interest is in continuing to work in the defense industry or government agencies by applying the skills he learned from his education at the University of Colorado Boulder.